

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2016-08-10

constexpr for std::char_traits

I. Introduction and Motivation

There is a request in the "C++ Standard Library Active Issues List" for [The char_traits specializations should declare their length\(\), compare\(\), and find\(\) members constexpr](#). Currently the following code fails to compile:

```
#include <string_view>
// Compile time error:
// > error: constexpr variable 'service' must be initialized by a constant expression
// > constexpr string_view service = "HELLO WORD SERVICE";
// >
// > string_view:110:39: note: non-constexpr function 'length' cannot be used in a constant expression
//
constexpr string_view service = "HELLO WORD SERVICE";
```

This proposal attempts to solve the issue 2232 and make std::string_view usable in constant expressions.

std::basic_string_view uses from traits_type only the static member functions length, compare and find. Only those functions are affected by this proposal.

A proof of concept implementation with benchmark is available at: [constexpr char_traits](#).

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to the existing header <string> such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core language and it can be (and has been) implemented in standard C++.

III. Analysis of existing std::char_traits implementations.

libstdc++ and libc++ have implementations of std::char_traits<char16_t> and std::char_traits<char32_t> that do not use C functions or intrinsics. Those two std::char_traits could be made constexpr without any impact on performance.

libstdc++ uses __builtin_strlen, __builtin_memcmp and __builtin_memchr intrinsics for std::char_traits<char>. Those extensions are already usable in constant expressions.

libstdc++ uses C functions for std::char_traits<wchar_t>; libc++ uses C functions for std::char_traits<char> and std::char_traits<wchar_t>.

Implementations of standard library, C library and compilers may differ a lot, but according to the performance measures from ["Appendix"](#) following conclusions could be made for eglibc-2.19:

- char related intrinsics outperform C calls
- char related intrinsics significantly outperform hand-made naïve implementation
- hand-made naïve implementations may outperform C functions on short wchar_t strings

Assuming that the situation is mostly the same on different platforms, it seems reasonable to:

- use intrinsics to make std::char_traits<char> constexpr. Typically this brings performance improvement.
- create constexpr-usable intrinsics for std::char_traits<wchar_t> or leave it non-constexpr. Two wordings are provided in this paper: one that does not mark std::char_traits<wchar_t> functions with constexpr and another one that does.

IV. Proposed wording relative to the Working Draft N4604.

Note: Only one of the wordings may be accepted, not both.

A. Wording that affects all the `std::char_traits` including `std::char_traits<wchar_t>`

Note for editor: Add `constexpr` in all the 21.2.3.* [char.traits.specializations.*]:

```
static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
static constexpr size_t length(const char_type* s);
static constexpr const char_type* find(const char_type* s, size_t n, const char_type& a);
```

B. Wording that affects all the `std::char_traits` except `std::char_traits<wchar_t>`

Note for editor: Add `constexpr` in all the 21.2.3.* [char.traits.specializations.*] except 21.2.3.4 [char.traits.specializations.wchar.t]:

```
static constexpr int compare(const char_type* s1, const char_type* s2, size_t n);
static constexpr size_t length(const char_type* s);
static constexpr const char_type* find(const char_type* s, size_t n, const char_type& a);
```

C. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_constexpr_char_traits`.

V. Revision History

Revision 0:

- Initial proposal.

VI. References

[N4604] C++17 CD Ballot Document. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4604.pdf.

[Antony Polukhin] Proof of concept implementation. Available online at https://github.com/apolukhin/apolukhin.github.io/tree/master/papers/constexpr_char_traits/.

[LWG2232] The `char_traits` specializations should declare their `length()`, `compare()`, and `find()` members `constexpr`. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/lwg-active.html#2232>.

VII. Acknowledgements

Walter E. Brown provided corrections and suggestions for this proposal.

VIII. Appendix.

Below are results of performance measures for different implementations of `char_traits`. Full output of benchmark and source codes are available online at https://github.com/apolukhin/apolukhin.github.io/tree/master/papers/constexpr_char_traits/.

First column in output describes the length of each char array that was given to the trait's function. Second and third columns show the median value of 500 runs of trait's function on 10000 char arrays in microseconds. Column "Relation" is equal to the second column divided on the third column. Last column is equal to *max(mean absolute deviation of first trait's function runs, mean absolute deviation of second trait's function runs)*.

Clang results:

TypeTrait::length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
String length				
10	40	48	1.20	+/-0.00
20	64	67	1.05	+/-0.00
30	64	66	1.03	+/-0.00
40	64	66	1.03	+/-0.00
50	65	69	1.06	+/-0.00
500	234	233	1.00	+/-0.01
5000	285	285	1.00	+/-0.01
50000	335	333	0.99	+/-0.02

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	40	68	1.70	+/-0.00
20	63	131	2.08	+/-0.00
30	64	199	3.11	+/-0.00
40	64	274	4.28	+/-0.00
50	65	309	4.75	+/-0.00
500	226	651	2.88	+/-0.01
5000	287	713	2.48	+/-0.02
50000	342	781	2.28	+/-0.02

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	67	68	1.01	+/-0.00
20	129	92	0.71	+/-0.00
30	200	118	0.59	+/-0.00
40	272	147	0.54	+/-0.00
50	307	177	0.58	+/-0.01
500	911	796	0.87	+/-0.03
5000	1108	1000	0.90	+/-0.02
50000	1742	1526	0.88	+/-0.01

TypeTrait::find

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	32	49	1.53	+/-0.00
20	94	86	0.91	+/-0.00
30	94	87	0.93	+/-0.00
40	98	92	0.94	+/-0.00
50	103	100	0.97	+/-0.00
500	278	278	1.00	+/-0.00
5000	308	307	1.00	+/-0.02
50000	355	356	1.00	+/-0.03

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	32	85	2.66	+/-0.00
20	94	161	1.71	+/-0.00
30	94	236	2.51	+/-0.00
40	98	321	3.28	+/-0.00
50	103	363	3.52	+/-0.00
500	272	784	2.88	+/-0.01
5000	296	781	2.64	+/-0.03
50000	356	872	2.45	+/-0.04

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	86	120	1.40	+/-0.00
20	164	154	0.94	+/-0.00
30	241	198	0.82	+/-0.00
40	334	230	0.69	+/-0.00
50	377	277	0.73	+/-0.00
500	1126	937	0.83	+/-0.01
5000	1305	1102	0.84	+/-0.02
50000	1488	1305	0.88	+/-0.00

TypeTrait::find2

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	46	43	0.93	+/-0.00
20	79	82	1.04	+/-0.00
30	79	82	1.04	+/-0.00
40	84	88	1.05	+/-0.00
50	94	93	0.99	+/-0.00
500	251	252	1.00	+/-0.00
5000	331	331	1.00	+/-0.02
50000	379	378	1.00	+/-0.02

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	46	87	1.89	+/-0.00
20	79	161	2.04	+/-0.00
30	79	237	3.00	+/-0.00
40	84	343	4.08	+/-0.00
50	94	386	4.11	+/-0.00
500	250	784	3.14	+/-0.00
5000	323	793	2.46	+/-0.02
50000	388	886	2.28	+/-0.02

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	86	123	1.43	+/-0.00
20	163	160	0.98	+/-0.00
30	240	203	0.85	+/-0.00
40	350	234	0.67	+/-0.00

50	393	282	0.72	+/-0.00
500	1095	908	0.83	+/-0.01
5000	1248	1092	0.88	+/-0.03
50000	1476	1326	0.90	+/-0.01

TypeTrait::compare

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	59	40	0.68	+/-0.00
20	48	42	0.88	+/-0.00
30	51	45	0.88	+/-0.00
40	62	55	0.89	+/-0.00
50	72	61	0.85	+/-0.00
500	317	317	1.00	+/-0.00
5000	3569	3570	1.00	+/-0.00
50000	28745	28679	1.00	+/-0.00

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	59	95	1.61	+/-0.00
20	48	181	3.77	+/-0.00
30	51	267	5.24	+/-0.00
40	62	355	5.73	+/-0.00
50	72	441	6.12	+/-0.00
500	317	4487	14.15	+/-0.01
5000	3572	44655	12.50	+/-0.00
50000	28724	461979	16.08	+/-0.00

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	92	64	0.70	+/-0.00
20	168	122	0.73	+/-0.00
30	248	125	0.50	+/-0.00
40	332	154	0.46	+/-0.00
50	418	172	0.41	+/-0.00
500	4407	1572	0.36	+/-0.01
5000	43248	14056	0.33	+/-0.00
50000	431685	112438	0.26	+/-0.00

GCC results:

TypeTrait::length

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	34	40	1.18	+/-0.00
20	56	56	1.00	+/-0.00
30	56	56	1.00	+/-0.00
40	56	56	1.00	+/-0.00
50	60	60	1.00	+/-0.00
500	238	241	1.01	+/-0.00
5000	303	304	1.00	+/-0.01
50000	348	351	1.01	+/-0.03

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	34	60	1.76	+/-0.00
20	57	126	2.21	+/-0.00
30	56	194	3.46	+/-0.00
40	56	272	4.86	+/-0.00
50	60	299	4.98	+/-0.00
500	238	596	2.50	+/-0.00
5000	304	632	2.08	+/-0.01
50000	380	708	1.86	+/-0.04

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	62	79	1.27	+/-0.00
20	128	103	0.80	+/-0.00
30	190	124	0.65	+/-0.01
40	273	152	0.56	+/-0.00
50	294	180	0.61	+/-0.01
500	822	839	1.02	+/-0.00
5000	1004	1000	1.00	+/-0.02
50000	1643	1538	0.94	+/-0.01

TypeTrait::find

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	49	42	0.86	+/-0.00
20	70	70	1.00	+/-0.00
30	70	70	1.00	+/-0.00
40	82	77	0.94	+/-0.00

50	94	96	1.02	+/-0.00
500	284	285	1.00	+/-0.00
5000	313	311	0.99	+/-0.03
50000	362	360	0.99	+/-0.04

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	49	83	1.69	+/-0.00
20	70	145	2.07	+/-0.00
30	70	212	3.03	+/-0.00
40	82	267	3.26	+/-0.00
50	94	330	3.51	+/-0.00
500	281	772	2.75	+/-0.00
5000	307	777	2.53	+/-0.04
50000	369	893	2.42	+/-0.05

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	100	174	1.74	+/-0.00
20	180	208	1.16	+/-0.00
30	262	255	0.97	+/-0.00
40	342	295	0.86	+/-0.00
50	391	347	0.89	+/-0.00
500	1119	1103	0.99	+/-0.01
5000	1300	1335	1.03	+/-0.01
50000	1542	1582	1.03	+/-0.01

TypeTrait::find2

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	33	39	1.18	+/-0.00
20	77	86	1.12	+/-0.00
30	77	86	1.12	+/-0.00
40	88	91	1.03	+/-0.00
50	92	97	1.05	+/-0.00
500	272	274	1.01	+/-0.00
5000	339	337	0.99	+/-0.02
50000	389	386	0.99	+/-0.02

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	33	85	2.58	+/-0.00
20	77	161	2.09	+/-0.00
30	77	242	3.14	+/-0.00
40	88	321	3.65	+/-0.00
50	92	371	4.03	+/-0.00
500	264	801	3.03	+/-0.00
5000	340	811	2.39	+/-0.02
50000	384	869	2.26	+/-0.03

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	84	116	1.38	+/-0.00
20	162	150	0.93	+/-0.00
30	240	194	0.81	+/-0.00
40	322	225	0.70	+/-0.00
50	373	274	0.73	+/-0.00
500	1073	901	0.84	+/-0.01
5000	1233	1102	0.89	+/-0.01
50000	1481	1356	0.92	+/-0.02

TypeTrait::compare

String length	intrinsics<char>	c_calls<char>	Relation	+/-deviation
10	54	40	0.74	+/-0.00
20	47	47	1.00	+/-0.00
30	50	43	0.86	+/-0.00
40	54	54	1.00	+/-0.00
50	70	56	0.80	+/-0.00
500	316	318	1.01	+/-0.00
5000	3589	3503	0.98	+/-0.01
50000	28598	28577	1.00	+/-0.00

String length	intrinsics<char>	hand-made<char>	Relation	+/-deviation
10	54	95	1.76	+/-0.00
20	47	184	3.91	+/-0.00
30	50	273	5.46	+/-0.00
40	54	399	7.39	+/-0.00
50	70	476	6.80	+/-0.00
500	314	3431	10.93	+/-0.00
5000	3590	33538	9.34	+/-0.00
50000	28582	333971	11.68	+/-0.00

String length	hand-made<wchar_t>	std::char_traits<wchar_t>	Relation	+/-deviation
10	113	57	0.50	+/-0.00
20	210	121	0.58	+/-0.00
30	307	119	0.39	+/-0.00
40	444	158	0.36	+/-0.00
50	509	174	0.34	+/-0.00
500	3832	1623	0.42	+/-0.00
5000	36665	14117	0.39	+/-0.00
50000	364886	112373	0.31	+/-0.00